

Preguntas sobre Reinforcement Learning

1. Preguntas sobre Gymnasium

1. ¿Cuáles son los 5 retornos de `env.step()`?

```
1 observation, reward, terminated, truncated, info = env.step(action)
```

— Enumerá y explicá brevemente el propósito de cada uno.

2. ¿Qué información contiene `env.observation_space` cuando es un `Box`?

```
1 print(env.observation_space)
2 # Ej: Box(low=array(...), high=array(...), shape=(4,), dtype=float32)
```

— ¿Qué nos dice `.low`, `.high`, `.shape` y `.dtype`?

3. ¿Qué ocurre si hacés `env.step()` tras un paso con `terminated=True` sin llamar a `reset()`?

4. ¿Qué devuelve cada llamada a `env.reset()`?

```
1 a, b = env.reset(seed=42)
```

— ¿Qué representan cada uno (`a` y `b`)?

5. ¿Para qué sirve pasar `seed` a `env.reset(seed=...)`?

— Explicá su utilidad.

6. Al crear el ambiente, ¿qué efecto tiene el parámetro `render_mode`?

```
1 env = gymnasium.make("CartPole-v1", render_mode="rgb_array")
```

7. Verdadero o Falso: “Gymnasium solo permite usar ambientes incluidos en su librería.”

8. ¿Para qué sirven los wrappers en Gymnasium?

— Escribir dos ejemplos reales que usaron: ¿qué hace cada uno?

9. ¿Qué error hay en este bucle de juego?

```

1 obs, info = env.reset()
2 for _ in range(500):
3     obs, reward, terminated, truncated, info = env.step(env.
4         action_space.sample())
5     if terminated:
6         obs, info = env.reset()

```

10. ¿Cómo limitarías manualmente un episodio a 50 pasos si el ambiente no tiene time limit?

2. Preguntas prácticas sobre K-bandits

1. Selección de acción greedy

Supongamos que tenés un array Q de tamaño K (valores estimados). ¿Cómo seleccionarías la acción greedy en código numpy?

2. Implementación de ϵ -greedy (pseudocódigo)

Escribí un fragmento de código o pseudocódigo para elegir una acción bajo la política ϵ -greedy, usando Q , ϵ , np.random.rand() .

3. Cálculo de ϵ annealado

Dado ϵ_{start} , ϵ_{end} , ϵ_{anneal} , y t (pasos actuales), escribí el código o pseudocódigo para calcular el valor de ϵ en el paso t .

4. Comentario de error en actualización de Q

```

1 q[arm] = reward

```

Comentá por qué esta línea está mal para aprendizaje incremental de Q , y qué implementación se usaría en su lugar.

5. Uso del array N

En lugar de guardar todos los rewards para cada brazo (una lista), usamos un array N (veces que se eligió cada brazo). ¿Por qué? ¿Cuál es su utilidad en el cálculo del nuevo $Q[\text{arm}]$? Escribir la actualización $Q[\text{arm}] = \dots$.

6. ¿Qué pasa si usamos un α constante?

Reescribí la actualización anterior para usar una tasa de aprendizaje constante α en lugar de $1/N[\text{arm}]$. ¿Cómo se modificaría la línea de actualización?

7. Inicialización optimista

Si en lugar de $Q = \text{np.zeros}(K)$ usamos $Q = \text{np.ones}(K) * \text{optimistic_value}$, ¿qué efecto práctico tiene en la exploración?

3. Programación Dinámica

En la notebook representamos la política como:

```

1 pi = {
2     (state, a): probabilidad
3 }

```

Si el espacio de acciones es discreto (`n_actions`), escribí un fragmento de código para muestrear una acción en base a las probabilidades de la política, dado un estado `s1`.

4. Monte Carlo

Dado el siguiente código:

```

1 def monte_carlo_prediction(policy, env, number_episodes, gamma
  =1.0):
2     V = defaultdict(float)
3     returns = defaultdict(lambda: [])
4     for _ in range(number_episodes):
5         episode = generate_episode(policy, env)
6         T = len(episode)
7         t = T
8         G = 0
9         states = [state for state, _, _, _ in episode]
10        for state, _, reward, _ in episode[::-1]:
11            t -= 1
12            G = (gamma * G) + reward
13            if not state in states[:t]:
14                returns[state].append(G)
15                V[state] = np.mean(returns[state])
16        return V

```

- ¿El código anterior implementa **First-Visit** o **Every-Visit**? Justificá.
- ¿Cómo lo modificarías para implementar el otro caso?

5. Temporal Difference

- En el MountainCar, ¿por qué era necesario discretizar las observaciones antes de usarlas para los algoritmos TD?
- ¿En qué rango de valores queda cada dimensión discretizada usando el siguiente código si `NUMBER_STATES = 20`?

```

1 def discretize_obs(obs):
2     env_low = env.observation_space.low
3     env_high = env.observation_space.high
4     env_den = (env_high - env_low) / NUMBER_STATES
5     pos_den = env_den[0]
6     vel_den = env_den[1]
7     pos_low = env_low[0]
8     vel_low = env_low[1]
9     pos_scaled = int((obs[0] - pos_low) / pos_den)

```

```

10     vel_scaled = int((obs[1] - vel_low) / vel_den)
11     return pos_scaled, vel_scaled

```

3. Para implementar la actualización Q-learning:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_{a \in \mathcal{A}(S')} Q(S', a) - Q(S, A) \right]$$

Supongamos que:

```

1 Q = np.zeros((NUMBER_STATES, NUMBER_STATES, env.action_space.
    n))

```

¿Cómo obtenés la máxima acción ($\max_a Q(S', a)$) en código para un par de índices (pos, vel) de un estado siguiente? Escribir el fragmento completo para la actualización de Q-learning dentro de un ciclo de entrenamiento, usando numpy.

```

1 for _ in tqdm(range(num_episodes)):
2     (pos, vel), _ = env.reset()
3     done = False
4     episode_reward = 0
5     while not done:
6         # Epsilon-greedy action selection
7         if np.random.rand() < epsilon:
8             action = env.action_space.sample() # Explore
9         else:
10            action = np.argmax(Q[pos, vel]) # Exploit
11
12        # Take action
13        (next_pos, next_vel), reward, terminated, truncated,
14        info = env.step(action)
15        episode_reward += reward
16
17        # Update Q-value
18        best_next_action = ...
19        Q[pos, vel, action] += ...
20
21        # Update state
22        (pos, vel) = (next_pos, next_vel)
23        done = terminated or truncated

```

6. PyTorch

1. Tensor vs Numpy array

¿En qué se diferencian un `torch.Tensor` y un `np.ndarray`?

2. Propiedades de Tensor

Listá y describí brevemente las propiedades de un tensor:

- dtype

- `shape`
- `ndim`
- `device`
- `requires_grad`

3. Argumentos clave de `nn.Linear`

- `nn.Linear(in_features, out_features)`
— ¿Qué representa cada argumento y cómo define el comportamiento de la capa?

4. Argumentos clave de `nn.Conv2d`

- `nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0)`
— Explicá el rol de cada argumento.

5. Código de entrenamiento – qué hace cada línea

```

1 images = images.to(DEVICE)
2 labels = labels.to(DEVICE)
3
4 optimizer.zero_grad()
5
6 predictions = model(images)
7
8 loss = loss_func(predictions, labels)
9
10 loss.backward()
11 optimizer.step()

```

— Explicá brevemente qué hace cada línea y por qué es necesaria en un ciclo de entrenamiento.